

Estructuras de Datos, 2026-2

Proyecto del curso

Avance: viernes 30 de octubre de 2026

Entrega final: lunes 16 de noviembre de 2026, 23:59

Sustentaciones: 18 y 25 de noviembre

De qué se trata

El proyecto se hace en grupos de **hasta tres estudiantes** y pide resolver un problema real escogiendo las estructuras de datos, implementando una de ellas desde cero y midiendo si lo que cuesta en la práctica coincide con lo que dice el análisis. Vale el 30 % del curso: el avance 10 y la entrega final con sustentación 20.

Hay que escoger uno de los cuatro escenarios de la sección siguiente. Cada escenario nombra dos estructuras que resuelven el mismo problema con costos distintos; el proyecto las implementa, las compara midiendo y explica la diferencia con lo visto en clase.

Los cuatro escenarios

- A. Inventario de una tienda.** Cada artículo tiene código, nombre y cantidad. Operaciones: agregar, eliminar por código, consultar por código, listar todo y reponer existencias. *Se comparan:* una lista enlazada escrita desde cero contra **vector** de la biblioteca. La pregunta que se mide: qué cuesta eliminar e insertar en medio cuando el inventario crece, y qué cuesta recorrerlo.
- B. Frecuencias de palabras de un texto.** Se lee un texto grande y se cuenta cuántas veces aparece cada palabra distinta. Operaciones: insertar o incrementar, consultar una palabra y listar las k más frecuentes. *Se comparan:* una tabla hash con encadenamiento escrita desde cero contra **map** de la biblioteca. La pregunta que se mide: cómo cambian las colisiones y el tiempo de consulta al variar el tamaño de la tabla y el factor de carga.
- C. Turnos de una clínica.** Los pacientes llegan, esperan y se atienden; los urgentes pasan antes que los demás. Operaciones: registrar llegada, atender al siguiente, consultar cuántos esperan y el tiempo promedio de espera. *Se comparan:* una cola circular escrita desde cero, que atiende por orden de llegada, contra una cola de prioridad. La pregunta que se mide: cómo cambia el tiempo de espera de los urgentes y qué cuesta cada operación en los dos casos.
- D. Índice de títulos.** Un catálogo de títulos de libros o de canciones que se consulta por nombre. Operaciones: insertar, buscar exacto, eliminar y listar en orden alfabético. *Se comparan:* un árbol binario de búsqueda escrito desde cero contra **set** de la biblioteca. La pregunta que se mide: cómo cambia la altura del árbol según el orden en que se insertan los títulos, y qué le pasa a la búsqueda cuando el árbol degenera.

La estructura escrita desde cero es la primera de cada pareja y no puede delegar en la biblioteca estándar: se implementa con arreglos, punteros o nodos, según corresponda.

Qué se entrega

El trabajo vive en un repositorio de GitHub, uno por grupo, que sale de *fork* del repositorio base: <https://github.com/EjerciciosClasesCardel/estr-2026-2-proyecto>.

1. Un integrante le hace fork; el fork queda público y las dos entregas se hacen sobre él. **Una entrega que no sea fork de ese repositorio pierde el 30 % de la nota.**
2. En **Settings > Collaborators** agrega a los demás integrantes, para que cada uno confirme con su propia cuenta.
3. Llenan el **README.md** con los integrantes y el escenario escogido.

Se califica el último commit anterior a la hora de cierre de cada entrega; lo que se suba después no se tiene en cuenta. La historia de commits cuenta doble: dice si hubo avance — un repositorio con un solo commit el día del cierre no lo muestra — y dice quién hizo qué. Los cambios tienen que ser **significativos**: los de puro formato o de comentarios no cuentan. **Cada integrante confirma con su propia cuenta**; un proyecto donde todo lo subió una sola persona se sustenta igual, pero la nota individual puede quedar distinta.

Avance — viernes 30 de octubre

1. **Informe corto** en **docs/informe-avance.md**: el escenario escogido descrito en sus propias palabras; qué guarda la estructura y qué operaciones necesita; por qué escogieron esa estructura y qué costo esperan de cada operación, citando el texto guía; y qué piensan medir.
2. **La estructura escrita desde cero**, con sus operaciones principales funcionando (en código/).
3. **Casos de prueba**: al menos tres funcionales y uno límite (estructura vacía, un solo elemento, estructura llena, clave repetida), con la salida esperada y la obtenida.
4. **Bitácora** (**bitacora.md**): una entrada fechada por sesión de trabajo, con qué hizo y qué quedó pendiente.

Entrega final — lunes 16 de noviembre, 23:59

1. **Las dos estructuras** resolviendo el mismo problema, con la misma interfaz de operaciones.
2. **Análisis formal** del costo en tiempo y en espacio de cada operación, con la estructura de cuatro partes: teorema, estrategia, desarrollo y conclusión con testigos c y k . Los invariantes se escriben como fórmulas.
3. **Experimentos**: midan las dos estructuras con entradas de tamaños crecientes, al menos cinco tamaños, con la misma máquina y el mismo compilador. Reporten los tiempos en una tabla y en una gráfica de Mermaid, y digan si lo medido coincide con lo que predijo el análisis; cuando no coincida, expliquen por qué.
4. **Informe final** en **docs/informe-final.md**: el del avance corregido, más el análisis, los experimentos y las conclusiones. Las estructuras se dibujan con Mermaid, no se describen en palabras.
5. **Sustentación** de 10 minutos: se explica el modelado, se muestra el código corriendo y se responden preguntas sobre las decisiones y los costos.

Reglas del repositorio

1. **Solo archivos de texto plano:** código fuente (`.c`, `.cpp`, `.h`), documentación (`.md`), datos de prueba (`.in`, `.out`, `.csv`) y el `Makefile`. No se aceptan comprimidos (`.zip`, `.rar`, `.tar.gz`) ni binarios (`.pdf`, `.docx`, `.xlsx`, ejecutables, imágenes escaneadas o insertadas). Los archivos de compilación se ignoran con `.gitignore`.
2. El `README.md` lleva el **nombre, el código y el correo institucional** de cada integrante, y el escenario escogido. Si el archivo falta, o si le falta alguno de esos datos, la entrega pierde el **20 %** de la nota. Quien no aparezca ahí no cuenta como integrante y su nota del proyecto es 0,0.
3. Las **primeras líneas de cada archivo de código** llevan los autores con su código: `// Autores: Nombre1 Codigo1, Nombre2 Codigo2`
4. Los informes van en **Markdown** dentro de la carpeta `docs/`, con los nombres `informe-avance.md` e `informe-final.md`. No se acepta otro formato ni archivos por fuera de esa carpeta.
5. La notación matemática se escribe en $\text{\LaTeX}$ dentro del Markdown, con los delimitadores `$...$` y `$$...$$`: las cotas, los testigos y los invariantes van así. No se aceptan imágenes de fórmulas.
6. **Todos los diagramas** — nodos y punteros de la estructura, árboles, tablas con sus colisiones, trazas de una operación, y la gráfica de los tiempos medidos — se hacen con **Mermaid** dentro del Markdown. La gráfica de los experimentos usa `xychart-beta`. No se aceptan imágenes insertadas ni diagramas escaneados.

Las tres guías de GitHub, en el orden de las reglas 4, 5 y 6:

- <https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>
- <https://docs.github.com/es/get-started/writing-on-github/working-with-advanced-formatting/writing-mathematical-expressions>
- <https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams>

Cómo se califica

Las dos entregas se califican sobre 5,0. El avance vale el 10 % del curso y la final con la sustentación, el 20 %. Todos los criterios son del grupo salvo la sustentación, que se pregunta a cada integrante y se califica por separado: la nota de un proyecto puede quedar distinta entre los tres si el trabajo o el dominio del tema lo estuvo.

Entrega	Criterio	Puntos
Avance	Modelado: el escenario, las operaciones y la estructura escogida, con la justificación y los costos esperados	30
	Implementación desde cero: las operaciones principales funcionan y respetan las restricciones de código	40
	Casos de prueba, incluido el límite, con salida esperada y obtenida	20
	Bitácora con entradas fechadas	10
Final	Las dos estructuras completas y con la misma interfaz	30
	Análisis formal de complejidad en tiempo y espacio	20
	Experimentos, con tabla, gráfica y contraste contra el análisis	20
	Informe: claridad, diagramas, citas al texto guía	15
	Sustentación, calificada por integrante	15

Quien no sustente recibe, como máximo, la quinta parte de la nota de la entrega final, aunque el resto del grupo sustente. Un proyecto cuyo código no compila no recibe los puntos de implementación ni los de experimentos.

Situación	Penalización
El repositorio entregado no es fork del repositorio base	30 % de la nota
El <code>README.md</code> falta o le falta el nombre, el código o el correo de algún integrante	20 % de la nota
Hay comprimidos o binarios en el repositorio	10 % de la nota
El informe no está en <code>docs/</code> y en Markdown	no recibe los puntos de informe

Restricciones de código

Las mismas del curso, y se califican:

- C o C++, compilado con `gcc -Wall -Wextra` o `g++ -Wall -Wextra`, sin advertencias.
- Solo biblioteca estándar. La estructura escrita desde cero no usa contenedores de la STL por dentro.
- Sin `break`, sin `continue`, sin `goto`, y un solo `return` al final de cada función.
- Toda reserva con `malloc` o `new` va con su liberación, y el retorno de `malloc` se verifica.
- El código compila con el `Makefile` del repositorio: `make` produce los ejecutables y `make pruebas` corre los casos.

Integridad

Dentro del grupo se comparte todo; entre grupos, no. Las entregas se comparan con herramientas de similitud y con un detector de patrones de código generado; lo escrito íntegramente por inteligencia artificial se considera plagio y puede derivar en un proceso ante la Dirección de Programa. Citen en el informe lo que consultaron.